Section 5.4 introduces one of the big features added to minimal APIs in .NET 7: filters. You can use filters to build a mini pipeline (similar to the middleware pipeline from chapter 4) for each of your endpoints. Like middleware, filters are great for extracting common code from your endpoint handlers, making your handlers easier to read.

You'll learn about the other big .NET 7 feature for minimal APIs in section 5.5: *route groups*. You can use route groups to reduce the duplication in your minimal APIs, extracting common routing prefixes and filters, making your APIs easier to read, and reducing boilerplate. In conjunction with filters, route groups address many of the common complaints raised against minimal APIs when they were released in .NET 6.

One great aspect of ASP.NET Core is the variety of applications you can create with it. The ability to easily build a generalized HTTP API presents the possibility of using ASP.NET Core in a greater range of situations than can be achieved with traditional web apps alone. But *should* you build an HTTP API, and if so, why? In the first section of this chapter, I'll go over some of the reasons why you may—or may not—want to create a web API.

## 5.1 What is an HTTP API, and when should you use one?

Traditional web applications handle requests by returning HTML, which is displayed to the user in a web browser. You can easily build applications like that by using Razor Pages to generate HTML with Razor templates, as you'll learn in part 2 of this book. This approach is common and well understood, but the modern application developer has other possibilities to consider (figure 5.1), as you first saw in chapter 2.